DSU.cpp

```
struct DSU {
    vi parent;
    vi size;
    DSU(int n) {
        parent.resize(n);
        size.resize(n, 1);
        for (int i = 0; i < n; i++)
            parent[i] = i;
    }
    // returns the leader of the set with element v
    int find_set(int v) {
        if (v == parent[v])
            return v;
        return parent[v] = find_set(parent[v]);
    }

    void union_sets(int a, int b) {
        a = find_set(a);
        b = find_set(b);
        if (a != b) {
            if (size[a] < size[b])
                swap(a, b);
            parent[b] = a;
            size[a] += size[b];
        }
    }
    int get_size(int a) {
        return size[find_set(a)];
    }
};
```

convex-hull.cpp

```cpp
// CREDIT: CP Algorithms
struct pt {
    double x, y;
    bool operator == (pt const& t) const {
        return x == t.x && y == t.y;
    }
};

int orientation(pt a, pt b, pt c) {
    double v = a.x*(b.y-c.y)+b.x*(c.y-a.y)+c.x*(a.y-b.y);
    if (v < 0) return -1; // clockwise
    if (v > 0) return +1; // counter-clockwise
    return 0;
}

bool cw(pt a, pt b, pt c, bool include_collinear) {
    int o = orientation(a, b, c);
    return o < 0 || (include_collinear && o == 0);
}
bool collinear(pt a, pt b, pt c) { return orientation(a, b, c) == 0; }

void convex_hull(vector<pt>& a, bool include_collinear = false) {
    pt p0 = *min_element(a.begin(), a.end(), [](pt a, pt b) {
        return make_pair(a.y, a.x) < make_pair(b.y, b.x);
    });
    sort(a.begin(), a.end(), [&p0](const pt& a, const pt& b) {
        int o = orientation(p0, a, b);
        if (o == 0)
            return (p0.x-a.x)*(p0.x-a.x) + (p0.y-a.y)*(p0.y-a.y)
                < (p0.x-b.x)*(p0.x-b.x) + (p0.y-b.y)*(p0.y-b.y);
        return o < 0;
    });
    if (include_collinear) {
        int i = (int)a.size()-1;
        while (i >= 0 && collinear(p0, a[i], a.back())) i--;
        reverse(a.begin()+i+1, a.end());
    }

    vector<pt> st;
    for (int i = 0; i < (int)a.size(); i++) {
        while (st.size() > 1 && !cw(st[st.size()-2], st.back(), a[i], include_collinear))
            st.pop_back();
        st.push_back(a[i]);
    }

    if (include_collinear == false && st.size() == 2 && st[0] == st[1])
        st.pop_back();

    a = st;
}
```

main.cpp

```cpp
#pragma optimize("O3")

#include <bits/stdc++.h>
using namespace std;

typedef long long ll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<vi> vvi;
typedef vector<ll> vll;
typedef vector<vll> vvll;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

}
```

segtree.cpp

```cpp
#include <bits/stdc++.h>
using namespace std;

int n;
vector<int> tree;

// build the tree, given a[] is the input array
// call on the root vertex, v=1, t=0, tr=n-1
void build(vector<int> a, int v, int tl, int tr) {
    if (tl == tr) {
        tree[v] = a[tl];
    } else {
        int tm = (tl + tr) / 2;
        build(a, v*2, tl, tm);
        build(a, v*2+1, tm+1, tr);
        tree[v] = tree[v*2] + tree[v*2+1];
    }
}

// sum of range l (left) to r (right)
int sum(int v, int tl, int tr, int l, int r) {
    if (l > r)
        return 0;
    if (l == tl && r == tr) {
        return tree[v];
    }
    int tm = (tl + tr) / 2;
    return sum(v*2, tl, tm, l, min(r, tm))
           + sum(v*2+1, tm+1, tr, max(l, tm+1), r);
}

// update index pos with new_val
void update(int v, int tl, int tr, int pos, int new_val) {
    if (tl == tr) {
        tree[v] = new_val;
    } else {
        int tm = (tl + tr) / 2;
        if (pos <= tm)
            update(v*2, tl, tm, pos, new_val);
        else
            update(v*2+1, tm+1, tr, pos, new_val);
        tree[v] = tree[v*2] + tree[v*2+1];
    }
}

int main() {
    // ios::sync_with_stdio(false);
    // cin.tie(nullptr);
    int n; // input size;
    tree.assign((4 * n), 0); // we can store the tree as array of 4 * n
}
```